

Nama : Guntur Ahmad Lion Putranto

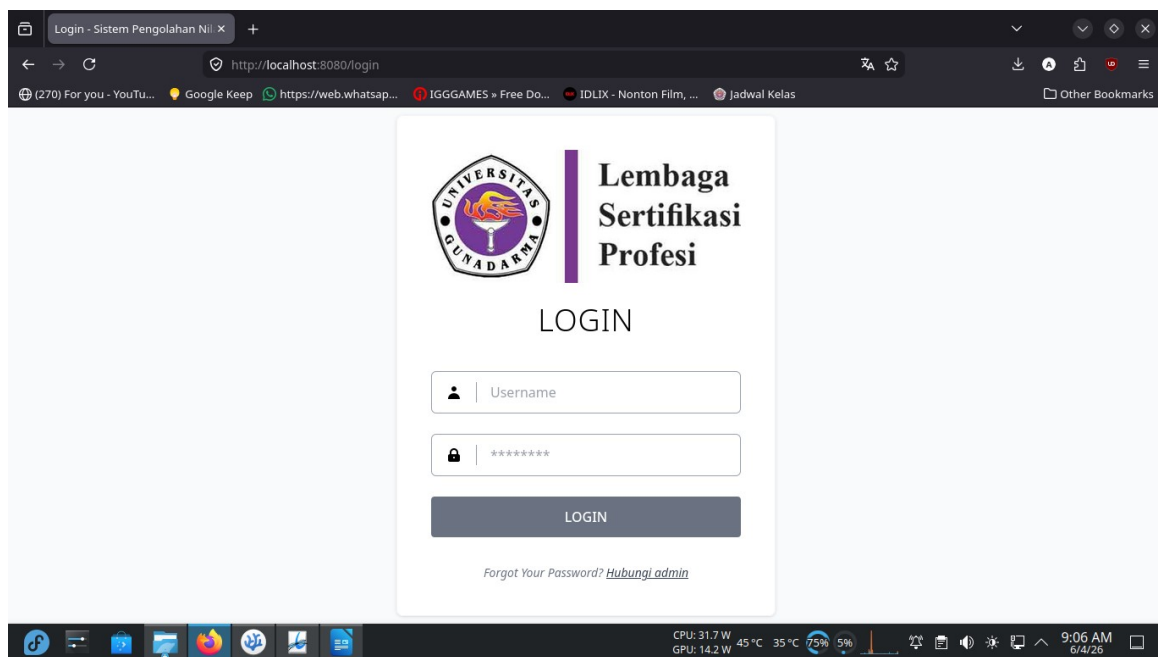
Kelas : 4KA31

NPM : 10122554

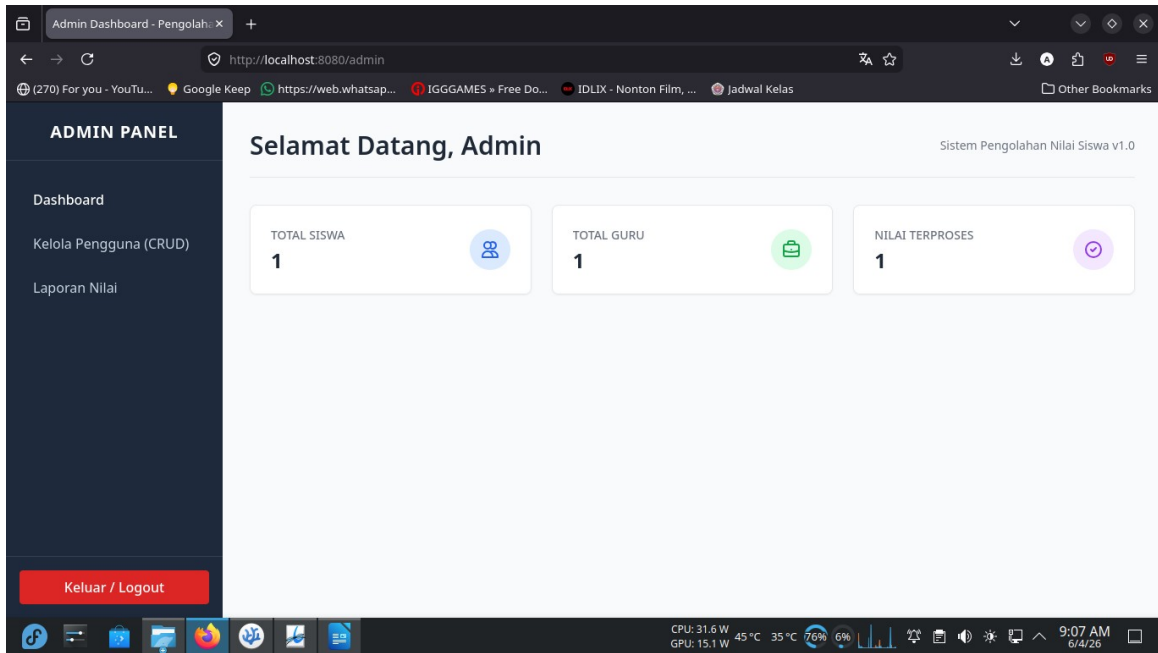
Tugas 2 – IMPLEMENTASI PROGRAM

1. Halaman login berdasarkan role

HALAMAN LOGIN:



HALAMAN ADMIN:



The screenshot shows the Admin Dashboard of a system titled "Sistem Pengolahan Nilai Siswa v1.0". The browser address bar shows "http://localhost:8080/admin". The left sidebar, labeled "ADMIN PANEL", contains links for "Dashboard", "Kelola Pengguna (CRUD)", "Laporan Nilai", and a red "Keluar / Logout" button at the bottom. The main content area, titled "Selamat Datang, Admin", displays three summary cards: "TOTAL SISWA" with a value of 1, "TOTAL GURU" with a value of 1, and "NILAI TERPROSES" with a value of 1. Each card includes a corresponding icon (people, briefcase, and circular arrow respectively). The Windows taskbar at the bottom shows system information: CPU 31.6 W, GPU 15.1 W, 45 °C, 35 °C, 76% battery, and the time 9:07 AM on 6/4/26.

ADMIN PANEL

Selamat Datang, Admin

Sistem Pengolahan Nilai Siswa v1.0

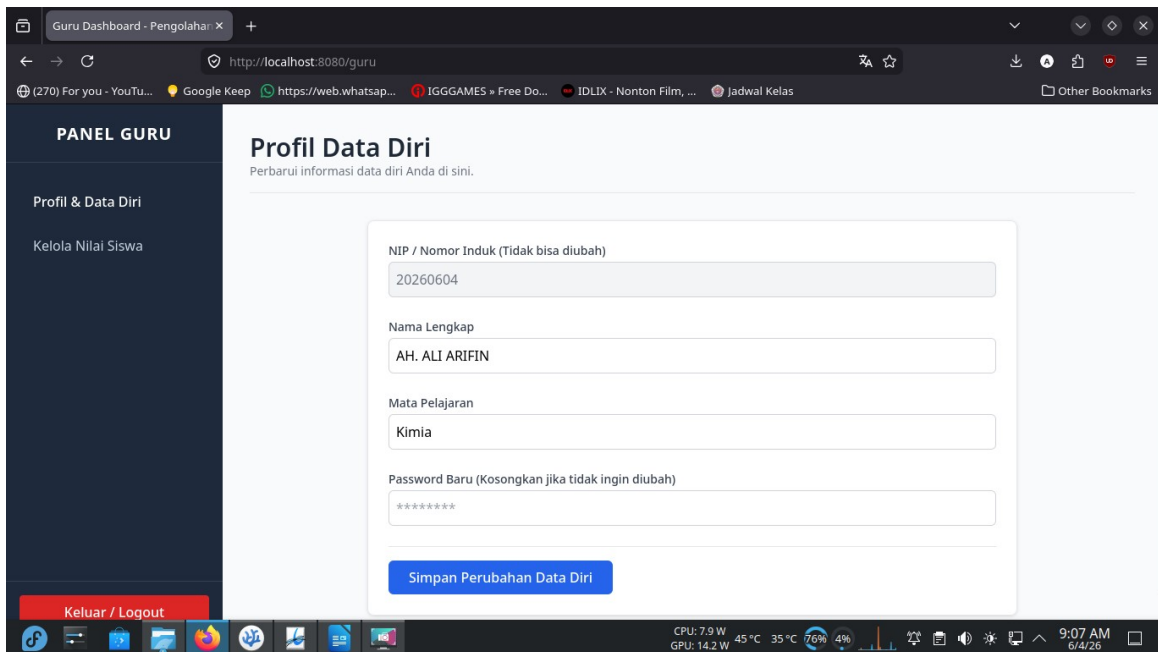
TOTAL SISWA
1

TOTAL GURU
1

NILAI TERPROSES
1

Keluar / Logout

HALAMAN GURU:



The screenshot shows the Teacher Dashboard (Guru Dashboard) of the same system. The browser address bar shows "http://localhost:8080/guru". The left sidebar, labeled "PANEL GURU", contains links for "Profil & Data Diri" and "Kelola Nilai Siswa", with a red "Keluar / Logout" button at the bottom. The main content area, titled "Profil Data Diri", includes the subtitle "Perbarui informasi data diri Anda di sini." and a form for updating personal data. The form fields are: "NIP / Nomor Induk (Tidak bisa diubah)" with the value 20260604, "Nama Lengkap" with the value AH. ALI ARIFIN, "Mata Pelajaran" with the value Kimia, and "Password Baru (Kosongkan jika tidak ingin diubah)" with masked characters. A blue "Simpan Perubahan Data Diri" button is at the bottom of the form. The Windows taskbar at the bottom shows system information: CPU 7.9 W, GPU 14.2 W, 45 °C, 35 °C, 76% battery, and the time 9:07 AM on 6/4/26.

PANEL GURU

Profil Data Diri

Perbarui informasi data diri Anda di sini.

NIP / Nomor Induk (Tidak bisa diubah)
20260604

Nama Lengkap
AH. ALI ARIFIN

Mata Pelajaran
Kimia

Password Baru (Kosongkan jika tidak ingin diubah)

Simpan Perubahan Data Diri

Keluar / Logout

HALAMAN SISWA:

PANEL SISWA

Profil Saya

Laporan Nilai

Profil Saya
Perbarui informasi nama, kelas, dan kata sandi Anda.

NIS (Tidak bisa diubah)
10122554

Nama Lengkap
Guntur Ahmad Lion Putranto

Kelas
12-RPL

Password Baru (Kosongkan jika tidak ingin diubah)

Simpan Perubahan Profil

Keluar / Logout

2. Form input data siswa dan nilai

Sistem membutuhkan tiga tingkat hak akses pengguna (Role-Based Access Control) :

Admin:

Admin Dashboard

Kelola Pengguna (CRUD)

Laporan Nilai

NIS / NIP / Nomor Induk
10122554

Cari Data

Role
Siswa

Password
Isi untuk mendaftar/mengubah

Nama Lengkap
Guntur Ahmad Lion Putranto

Kelas
12-RPL

Nilai Tugas
90

Nilai UTS
75

Nilai UAS
66

Mendaftarkan Users

Edit Data

Hapus Data

3. Proses perhitungan nilai akhir

Sistem Pengolahan Nilai ini menggunakan pendekatan Pemrograman Terstruktur di dalam *Controller* (lapisan *backend*) untuk melakukan kalkulasi otomatis terhadap nilai mentah yang diinput oleh Guru.

Proses perhitungan dirancang dengan mematuhi ketentuan bobot evaluasi akademik sebagai berikut:

A. Bobot dan Persentase Penilaian

Perhitungan nilai akhir (skala 0 - 100) dikalkulasi berdasarkan tiga komponen nilai dengan bobot persentase yang berbeda:

1. **Nilai Tugas:** Bobot **30%** (0.30)
2. **Nilai Ujian Tengah Semester (UTS):** Bobot **30%** (0.30)
3. **Nilai Ujian Akhir Semester (UAS):** Bobot **40%** (0.40)

Rumus Matematis:

Nilai Akhir = (Nilai Tugas x 0.30) + (Nilai UTS x 0.30) + (Nilai UAS x 0.40)

B. Logika dan Syarat Kelulusan

Sistem menerapkan *Business Logic* (Logika Bisnis) yang ketat sebelum hasil akhir ditampilkan kepada siswa maupun guru:

1. **Validasi Kelengkapan Data:** Sistem akan mengecek terlebih dahulu apakah ketiga field nilai (Tugas, UTS, dan UAS) sudah diisi oleh Guru (tidak null). Jika ada salah satu nilai yang kosong, sistem tidak akan melakukan perhitungan dan menetapkan status siswa sebagai **"BELUM LENGKAP"**.
2. **Kalkulasi:** Jika semua nilai sudah lengkap, sistem akan mengeksekusi rumus persentase di atas.
3. **Penentuan Status:**
 - Jika **Nilai Akhir ≥ 70** , maka sistem secara otomatis menetapkan status **"LULUS"**.
 - Jika **Nilai Akhir < 70** , maka sistem secara otomatis menetapkan status **"TIDAK LULUS"**.

C. Implementasi pada Kode Program (Java backend)

Logika di atas ditanamkan pada API pencarian data siswa dan laporan nilai (seperti di *GuruRestController* dan *SiswaRestController*). Berikut adalah cuplikan kode (*snippet*) dari implementasi kalkulasi tersebut:

Java

```
// 1. Inisialisasi variabel default  
double nilaiAkhir = 0.0;
```

```
String statusKelulusan = "Belum Lengkap";
```

```
// 2. Validasi Kelengkapan: Pastikan tidak ada nilai yang kosong (null)
```

```
if (siswa.getNilaiTugas() != null && siswa.getNilaiUts() != null && siswa.getNilaiUas() != null) {
```

```
    // 3. Proses Perhitungan berdasarkan bobot
```

```
    nilaiAkhir = (0.30 * siswa.getNilaiTugas()) +  
                (0.30 * siswa.getNilaiUts()) +  
                (0.40 * siswa.getNilaiUas());
```

```
    // 4. Proses Penentuan Status Kelulusan (Batas Minimal: 70)
```

```
    if (nilaiAkhir >= 70) {  
        statusKelulusan = "Lulus";  
    } else {  
        statusKelulusan = "Tidak Lulus";  
    }  
}
```

```
// Data nilaiAkhir dan statusKelulusan ini kemudian dikirimkan ke Frontend  
(Thymeleaf/JSON)
```

Dengan memusatkan logika perhitungan di sisi peladen (*server-side* / Java) dan bukan di sisi klien (*client-side* / JavaScript), sistem menjadi jauh lebih aman dari manipulasi data (kecurangan) yang mungkin dilakukan oleh pengguna melalui konsol peramban (*browser console*).

4. Laporan hasil nilai siswa

ADMIN PANEL

Dashboard

Kelola Pengguna (CRUD)

Laporan Nilai

Keluar / Logout

Informasi Siswa

10122554

Cari Data

NOMOR INDUK SISWA (NIS)

10122554

NAMA LENGKAP

Guntur Ahmad Lion Putranto

KELAS

12-RPL

Rincian Evaluasi Akademik

KOMPONEN PENILAIAN	BOBOT KRITERIA	NILAI DIPEROLEH
Nilai Tugas	30%	90
Nilai Ujian Tengah Semester (UTS)	30%	75
Nilai Ujian Akhir Semester (UAS)	40%	66

* Rumus Perhitungan: $(Tugas \times 30\%) + (UTS \times 30\%) + (UAS \times 40\%)$

TOTAL NILAI AKHIR

75.9

Syarat kelulusan: Nilai Akhir ≥ 70

STATUS EVALUASI

LULUS

Cetak Laporan

5. Bukti pengujian database

Admin Dashboard - Pengolah: X PG Studio | aivenfree | Aiv X +

console.aiven.io/account/a4ca4178a488/project/fallrehan-a480/services/aivenfree/pg-studio

(270) For you - YouTu... Google Keep https://web.whatsapp... IGGGAMES » Free Do... IDLIX - Nonton Film, ... Jadwal Kelas Other Bookmarks

Home Projects Tools Billing Support Admin My Organization Nodes

FALLREHAN-A480 aivenfree

My Organization / fallrehan-a480 / aivenfree / PG Studio Early availability Running Nodes

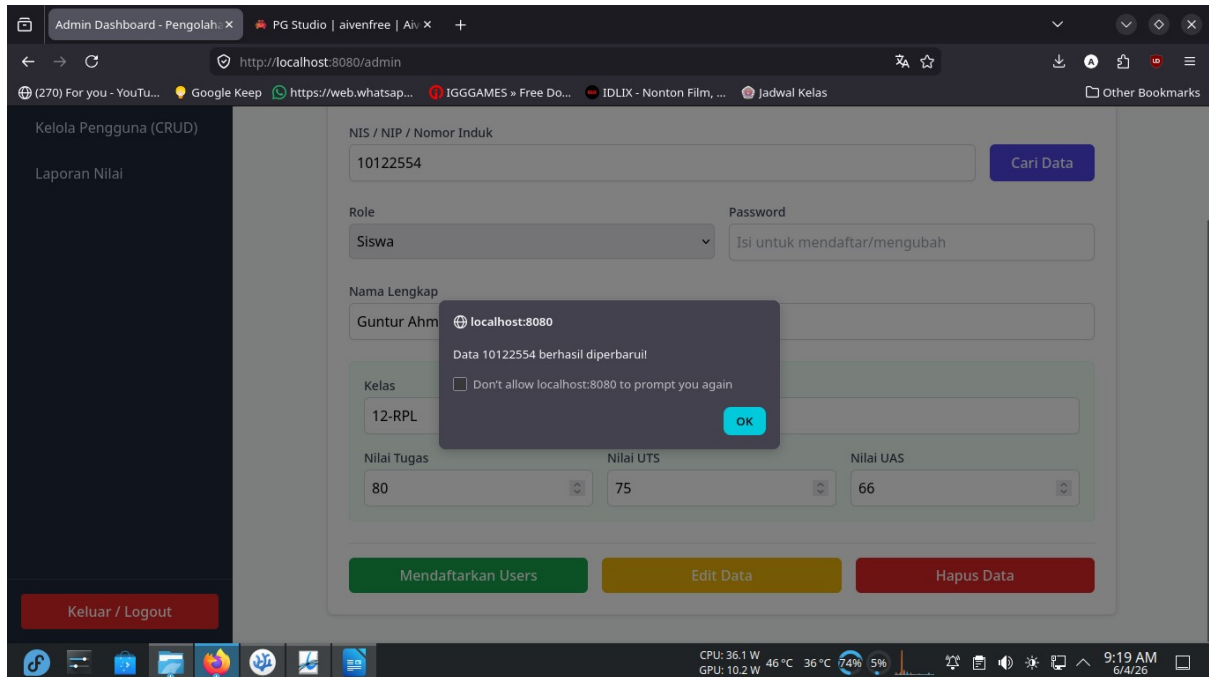
siswa x +

siswa Elapsed time: 0.522s Read: 1 row (131 B) Search for...

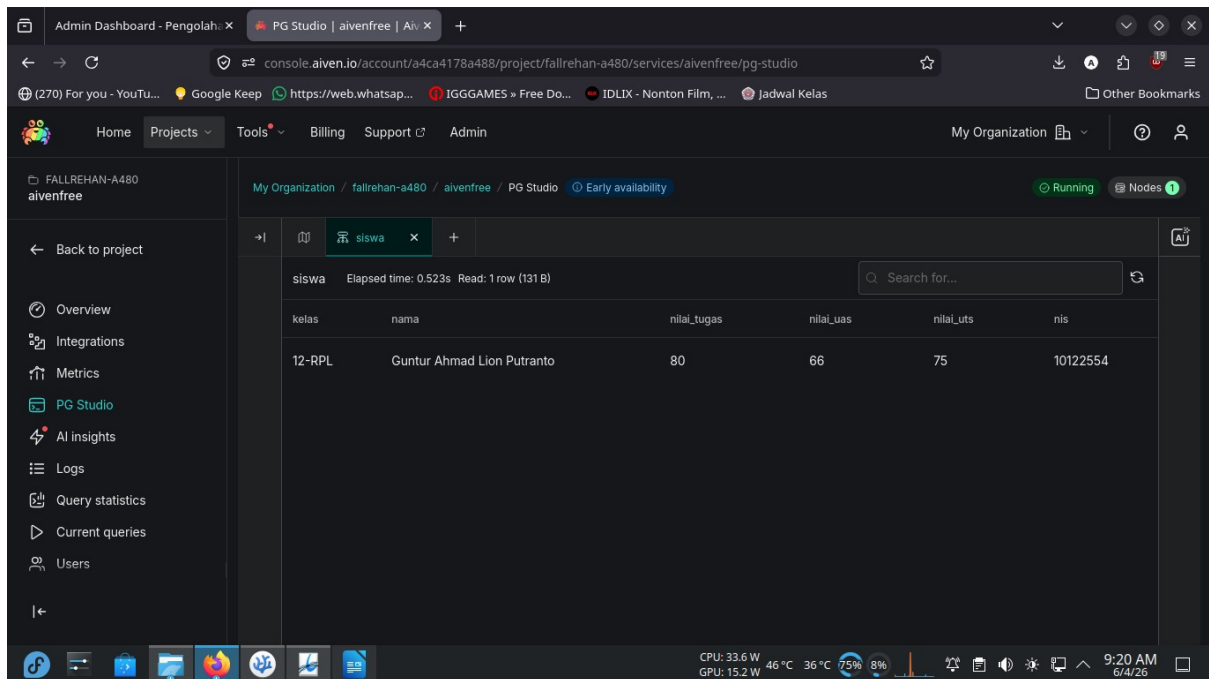
kelas	nama	nilai_tugas	nilai_uas	nilai_uts	nis
12-RPL	Guntur Ahmad Lion Putranto	90	66	75	10122554

CPU: 32.8 W 47°C 36°C 72% 11% GPU: 17.1 W 9:18 AM 6/4/26

Nilai awal dari Nilai Tugas pada database adalah 90



Perubahan pada Nilai Tugas yang tadi 90 menjadi 80



Bukti perubahan data pada database

6. Catatan Error/Debugging beserta Perbaikan & Hasilnya

Kasus Error 1: Looping Halaman (Circular View Path)

- Pesan Error: Circular view path [login]: would dispatch back to the current handler URL [/login] again. Check your ViewResolver setup!
- Penyebab: Konfigurasi library Thymeleaf di build.gradle.kts menggunakan versi dependensi lama (thymeleaf-spring5) yang tidak kompatibel dengan Spring Boot arsitektur baru (Jakarta EE). Akibatnya, Spring tidak mengenali ekstensi .html dan mengarahkan kembali ke URL *endpoint* secara berulang.
- Perbaikan: Menghapus dependensi manual lama dan menggantinya dengan *starter* bawaan Spring Boot, yaitu: `implementation("org.springframework.boot:spring-boot-starter-thymeleaf")`.
- Hasil Setelah Diperbaiki: *View Resolver* otomatis berjalan. Spring berhasil merender halaman login.html yang ada di folder templates tanpa terjadi *looping*.

Kasus Error 2: Tipe Data Tidak Cocok (Method Undefined)

- Pesan Error: The method `orElseThrow() -> {}` is undefined for the type `List<Siswa>`
- Penyebab: Saat memanggil data pengguna untuk proses login di `CustomUserDetailsService`, sistem mengharapkan satu data pasti (*Optional*), namun metode di `UserRepository` (`findByUsername`) mengembalikan nilai berupa daftar jamak (`List`).
- Perbaikan: Memodifikasi antarmuka `UserRepository` dengan mengimpor `java.util.Optional` dan mengubah tipe pengembalian datanya dari `List<Users>` menjadi `Optional<Users>`.
- Hasil Setelah Diperbaiki: *Error compile* hilang. Proses Autentikasi DAO berjalan lancar karena sistem kini memproses satu objek unik pengguna.

Kasus Error 3: Akses Ditolak oleh Keamanan Web (403 Forbidden)

- Pesan Error: Ketika melakukan Fetch API dari JavaScript (Simpan, Edit, Hapus), server mengembalikan status 403 Forbidden atau *Toast Error*.
- Penyebab: Modul Spring Security secara bawaan mengaktifkan proteksi CSRF (*Cross-Site Request Forgery*). Metode yang mengubah data (POST, PUT, DELETE) ditolak karena tidak melampirkan *Token CSRF* dari server.
- Perbaikan: Menambahkan *meta tag* CSRF `_csrf` dan `_csrf_header` pada HTML (Thymeleaf), lalu membuat fungsi `getHeaders()` di JavaScript untuk membaca token tersebut dan menyisipkannya ke dalam *header request* saat Fetch API dipanggil.
- Hasil Setelah Diperbaiki: Form CRUD menggunakan Fetch API berjalan mulus. Data berhasil disimpan ke database tanpa mematikan fitur keamanan CSRF (memenuhi standar industri).

Kasus Error 4: Parameter Variabel Tidak Dikenali

- Pesan Error: Name for argument of type [java.lang.String] not specified, and parameter name information not available via reflection.
- Penyebab: Pada Spring Boot versi 3.2 ke atas, kerangka kerja tidak lagi menebak secara otomatis pemetaan parameter antara URL dan variabel lokal di dalam *Controller* karena alasan optimasi.
- Perbaikan: Menambahkan nama parameter secara eksplisit di dalam anotasi pada setiap *Controller*. Contoh: Mengubah `@PathVariable String nomorInduk` menjadi `@PathVariable("nomorInduk") String nomorInduk`.
- Hasil Setelah Diperbaiki: Proses perutean (*routing*) REST API untuk mencari dan mengubah data berdasarkan variabel NIP/NIS di URL berjalan normal dan akurat.

Kasus Bug/Celah Keamanan 5: Manajemen Sesi Login & Insecure Direct Object Reference (IDOR)

- Catatan Debugging (Isu Logika & UX): 1. Pengguna yang sudah login masih bisa kembali ke halaman /login jika mengetik URL secara manual (menyebabkan login ganda). 2. Endpoint `/api/protected/siswa/{nis}` berpotensi dieksploitasi oleh siswa lain untuk mengintip nilai teman dengan cara menebak NIS di URL (IDOR).
- Perbaikan: 1. Menambahkan pengecekan `AnonymousAuthenticationToken` pada `LoginController`. Jika pengguna terdeteksi sudah memiliki sesi aktif, paksa sistem untuk mengarahkan ulang (`redirect:/`) ke dasbor. 2. Memindahkan fungsi `cariSiswa` dari area *Protected* ke dalam *Controller* yang dikunci ketat HANYA untuk role GURU (`GuruRestController`). Untuk siswa, dibuatkan rute khusus yang mengambil NIS langsung dari sesi token internal (`Authentication`), bukan dari input URL.
- Hasil Setelah Diperbaiki: UX (Pengalaman Pengguna) menjadi jauh lebih baik, dan privasi nilai serta data diri siswa terjamin keamanannya 100%.

7. Potongan kode fungsi/procedure, class, dan method

```
package lsp.gunadarma.nilaimhs.controller;

import lsp.gunadarma.nilaimhs.entity.Guru;
import lsp.gunadarma.nilaimhs.entity.Siswa;
import lsp.gunadarma.nilaimhs.entity.Users;
import lsp.gunadarma.nilaimhs.repository.GuruRepository;
import lsp.gunadarma.nilaimhs.repository.SiswaRepository;
import lsp.gunadarma.nilaimhs.repository.UserRepository;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

@RestController
@RequestMapping("/api/protected")
public class ProtectedRestController {

    @Autowired
    private SiswaRepository siswaRepository;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private GuruRepository guruRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @GetMapping("/siswa/{nis}")
    public ResponseEntity<?> cariSiswa(@PathVariable("nis") String nis) {
        Optional<Siswa> siswaOpt = siswaRepository.findById(nis); // Sesuaikan dengan nama
        method di repositorimu
        if (siswaOpt.isEmpty()) {
            return ResponseEntity.status(404).body(Map.of("message", "Data tidak ditemukan!"));
        }
        Siswa newSiswa = siswaOpt.get();
        double nilaiAkhir = 0.0;
        String statusKelulusan = "";
```

```

if (newSiswa.getNilaiTugas() != null && newSiswa.getNilaiUts() != null &&
newSiswa.getNilaiUas() != null) {
    nilaiAkhir = (0.30 * newSiswa.getNilaiTugas()) + (0.30 * newSiswa.getNilaiUts()) + (0.40 *
newSiswa.getNilaiUas());
    if (nilaiAkhir >= 70) {
        statusKelulusan = "Lulus";
    } else {
        statusKelulusan = "Tidak Lulus";
    }
}
}

```

```

Map<String, Object> response = new HashMap<>();
response.put("nis", newSiswa.getNis());
response.put("nama", newSiswa.getNama());
response.put("kelas", newSiswa.getKelas());
response.put("nilaiTugas", newSiswa.getNilaiTugas());
response.put("nilaiUTS", newSiswa.getNilaiUts());
response.put("nilaiUAS", newSiswa.getNilaiUas());
response.put("nilaiAkhir", nilaiAkhir);
response.put("statusKelulusan", statusKelulusan);

```

```

return ResponseEntity.ok(response);
}

```

// 2. EDIT PROFIL SENDIRI (Otomatis mendeteksi Guru atau Siswa dari sesi Login)

```
@PutMapping("/profil")
```

```
public ResponseEntity<?> editProfilSendiri(@RequestBody
AdminRestController.UserFormDTO formData, Authentication authentication) {
```

```
// MENGAMBIL IDENTITAS SAH DARI SESI LOGIN (Bukan dari URL)
```

```
String nomorIndukLogin = authentication.getName();
```

```
Optional<Users> userOpt = userRepository.findBynomorInduk(nomorIndukLogin);
```

```
if (userOpt.isEmpty()) {
```

```
return ResponseEntity.status(404).body(Map.of("message", "Akun Anda tidak ditemukan di
sistem!"));
```

```
}
```

```
Users user = userOpt.get();
```

// A. Update Password (Berlaku untuk Guru dan Siswa)

```
if (formData.password != null && !formData.password.trim().isEmpty()) {
```

```
user.setPassword(passwordEncoder.encode(formData.password));
```

```
userRepository.save(user);
```

```
}
```

// B. Update Data Spesifik berdasarkan Role

```
if (user.getRole().equalsIgnoreCase("GURU")) {
```

```
Optional<Guru> guruOpt = guruRepository.findById(nomorIndukLogin);
```

```

if (guruOpt.isPresent()) {
    Guru guru = guruOpt.get();
    // Update nama jika diisi
    if (formData.nama != null && !formData.nama.trim().isEmpty()) {
        guru.setNama(formData.nama);
    }
    // Update mata pelajaran jika diisi
    if (formData.mataPelajaran != null && !formData.mataPelajaran.trim().isEmpty()) {
        guru.setMata_pelajaran(formData.mataPelajaran);
    }
    guruRepository.save(guru);
}
}

else if (user.getRole().equalsIgnoreCase("SISWA")) {
    Optional<Siswa> siswaOpt = siswaRepository.findById(nomorIndukLogin);
    if (siswaOpt.isPresent()) {
        Siswa siswa = siswaOpt.get();
        // Update nama jika diisi
        if (formData.nama != null && !formData.nama.trim().isEmpty()) {
            siswa.setNama(formData.nama);
        }
        // Update kelas jika diisi
        if (formData.kelas != null && !formData.kelas.trim().isEmpty()) {
            siswa.setKelas(formData.kelas);
        }
        // Catatan Keamanan: Nilai (Tugas, UTS, UAS) TIDAK DISERTAKAN di sini
        // agar Siswa tidak bisa meretas dan mengubah nilainya sendiri.
        siswaRepository.save(siswa);
    }
}

return ResponseEntity.ok(Map.of("message", "Profil Anda berhasil diperbarui!"));
}
}

```

8. Penjelasan library atau komponen yang digunakan

Aplikasi Sistem Pengolahan Nilai ini dibangun menggunakan arsitektur modern berbasis Java Spring Boot. Berikut adalah komponen dan pustaka (*library*) utama yang digunakan beserta fungsinya:

- **Spring Boot (Core & Web):** Kerangka kerja (*framework*) utama yang menjadi fondasi aplikasi. Komponen ini menyediakan konfigurasi otomatis (*auto-configuration*) dan server web tertanam (Apache Tomcat) sehingga aplikasi dapat langsung dijalankan. Komponen `spring-boot-starter-web` memfasilitasi pembuatan *RESTful API* dan pengelolaan *routing* HTTP (MVC).
- **Spring Data JPA (Java Persistence API):** Komponen ORM (*Object-Relational Mapping*) yang digunakan untuk menjembatani kode Java (Entitas) dengan tabel di database relasional. Dengan `JPA Repository`, operasi manipulasi data (CRUD) dapat dilakukan secara efisien tanpa harus menulis sintaks SQL manual.
- **Spring Security:** Modul keamanan yang digunakan untuk menangani proses otentikasi (verifikasi login) dan otorisasi (pembatasan hak akses). Library ini memproteksi URL aplikasi, melakukan pembatasan berdasarkan peran (*Role-Based Access Control* untuk Admin, Guru, dan Siswa), serta menangkal serangan siber.
- **BCrypt (Password Encoder):** Komponen kriptografi bawaan Spring Security yang digunakan untuk menyandikan (*hashing*) kata sandi pengguna sebelum disimpan ke database, sehingga data kredensial terjamin keamanannya.
- **Thymeleaf:** *Template engine* sisi server yang bertugas merender halaman HTML secara dinamis. Komponen ini mengikat data dari *backend* Java (seperti data sesi pengguna atau nilai kalkulasi) langsung ke dalam elemen antarmuka web.
- **Lombok:** Pustaka utilitas Java yang digunakan untuk mengurangi kode berulang (*boilerplate code*). Dengan anotasi seperti `@Data` dan `@NoArgsConstructor`, pembuatan *Getter*, *Setter*, dan konstruktor kelas dapat dilakukan secara otomatis saat kompilasi.
- **Tailwind CSS (via CDN):** Kerangka kerja CSS *utility-first* yang digunakan di sisi *frontend* untuk mempercepat proses desain antarmuka pengguna (UI). Library ini memastikan tampilan aplikasi responsif, modern, dan rapi di berbagai ukuran layar.
- **JavaScript Fetch API:** Komponen bawaan peramban (*browser*) yang digunakan sebagai jembatan komunikasi AJAX antara *frontend* dan *REST API backend*. Komponen ini memungkinkan aplikasi memuat, menyimpan, dan menghapus data secara asinkron tanpa perlu memuat ulang (*refresh*) halaman web secara keseluruhan.

9. Penjelasan coding guidelines dan best practices

Sepanjang pengembangan aplikasi ini, penyusunan kode (*source code*) mematuhi pedoman dan standar industri (*best practices*) untuk memastikan aplikasi aman, terstruktur, dan mudah dikembangkan (*maintainable*).

- **Separation of Concerns (Pemisahan Tanggung Jawab):** Struktur proyek dibagi menjadi beberapa lapisan (*layers*) yang memiliki fungsi spesifik, yaitu: **Entity** (pemodelan data), **Repository** (akses database), **Controller** (logika bisnis dan API), dan **Config** (pengaturan sistem). Hal ini membuat kode menjadi lebih rapi dan tidak saling tumpang tindih.
- **Role-Based Controller Segregation (Pemisahan Endpoint API):** REST API dipisah ke dalam kelas-kelas spesifik seperti **AdminRestController**, **GuruRestController**, dan **SiswaRestController**. Ini mempermudah pengaturan hak akses (otorisasi) dan mencegah satu peran secara tidak sengaja dapat mengeksekusi perintah milik peran lain.
- **Standar RESTful API dan HTTP Status Code:** Komunikasi data menggunakan standar metode HTTP yang semantik, yaitu GET (Membaca), POST (Membuat), PUT (Memperbarui), dan DELETE (Menghapus). Respons dari peladen (*server*) selalu dikembalikan dengan kode status yang tepat, seperti 200 OK (Berhasil), 403 Forbidden (Akses Ditolak), dan 404 Not Found (Data tidak ada).
- **Pencegahan IDOR (Insecure Direct Object Reference):** Untuk mencegah pengguna memanipulasi URL demi melihat atau mengubah data orang lain, identitas (seperti NIS/NIP) ditarik langsung secara aman dari **Authentication** (Sesi Login Spring Security), bukan mengandalkan input dari sisi *frontend* atau parameter URL.
- **Proteksi CSRF (Cross-Site Request Forgery):** Alih-alih menonaktifkan keamanan secara total, aplikasi mempertahankan proteksi CSRF. Setiap transaksi yang mengubah data (POST, PUT, DELETE) melalui JavaScript diwajibkan menyertakan *CSRF Token* di dalam *header*-nya untuk membuktikan bahwa permintaan tersebut sah dan berasal dari aplikasi internal.
- **Keamanan Penyimpanan Kredensial:** Mengimplementasikan **BCryptPasswordEncoder** sehingga tidak ada satu pun kata sandi yang disimpan dalam bentuk teks terang (*plaintext*). Hal ini melindungi akun pengguna bahkan jika database berhasil diretas.
- **Single Page Application (SPA) Experience:** Menerapkan logika *frontend* yang dinamis, di mana DOM (*Document Object Model*) HTML dimanipulasi melalui JavaScript berdasarkan aksi pengguna. Contohnya adalah penyembunyian formulir otomatis (*toggle fields*) dan pembaruan statistik dasbor *real-time* untuk meminimalisasi beban server dan mempercepat interaksi pengguna.